

Euclid-MCP: A Model Context Protocol Server for Deterministic Logical Reasoning via Prolog

Bartolomeo Bogliolo

Abstract

Large Language Models (LLMs) excel at natural language understanding and generation but remain unreliable for multi-step logical reasoning, especially in safety-critical or compliance-sensitive domains. Recent neuro-symbolic approaches address this gap by coupling neural models with external symbolic engines, yet most integrations are bespoke and lack a standardized interface for tool-augmented agents. This paper presents EUCLID-MCP, an open-source MCP server that provides deterministic logical reasoning via SWI-Prolog.

EUCLID-MCP introduces EUCLID-IR, an engine-agnostic intermediate representation for Horn-clause logic that is human-readable, easy for LLMs to generate, and straightforward to compile into Prolog or alternative backends. The server exposes a compact tool interface that supports a translate-run-inspect-repair loop, enabling LLM clients to delegate inference while retaining full access to proof traces and derivation logs.

We evaluate EUCLID-MCP on a realistic IT security and compliance use case. Results show that while LLMs alone are sufficient on small knowledge bases, they hallucinate systematically on larger problems, whereas EUCLID-MCP delivers exact answers with lower latency and more compact outputs. We argue that semantic RAG is fundamentally unsuited for rule enforcement, and that EUCLID-MCP can serve as a stable, shared reasoning substrate for both RAG-based assistants and agentic systems.

1 Introduction

The rapid progress of Large Language Models (LLMs) has transformed many aspects of natural language processing, from question answering and summarization to code generation and agent-like interaction. However, when tasks require **multi-step logical deduction**, strict adherence to formal

rules, or **auditable decision traces**, purely neural approaches exhibit well-documented limitations: non-deterministic outputs, susceptibility to hallucination, and difficulty in providing faithful reasoning proofs. These issues are particularly acute in domains such as **business rule enforcement**, **regulatory compliance**, and **security policy verification**, where decisions must be both correct and explainable.

A growing body of research addresses these shortcomings through **neuro-symbolic AI**, which combines the pattern-recognition strengths of neural models with the rigor of symbolic reasoning. One prominent direction delegates inference to external solvers—such as SAT/SMT solvers, constraint programming systems, or logic programming engines—while relying on the LLM to translate informal problem descriptions into a formal representation. Recent work has shown that grounding LLM reasoning in **Prolog** can significantly improve both answer accuracy and the reliability of reasoning proofs [3, 4], provided the translation from natural language to logic is sufficiently accurate. At the same time, the emerging **Model Context Protocol (MCP)** offers a standardized way for LLM applications to discover and invoke external tools and data sources [5], but few existing MCP servers expose formal reasoning engines as first-class, reusable components.

Our project originated from the observation that retrieval-augmented generation (RAG) based on semantic similarity is fundamentally mismatched to rule-centric tasks, where decisions must follow logically from explicit policies rather than from approximate textual similarity. This paper introduces **EUCLID-MCP**, an open-source MCP server that provides **deterministic logical reasoning via Prolog** for any MCP-capable LLM client. **EUCLID-MCP** implements a hybrid cognitive architecture in which a lightweight LLM describes the world in terms of facts and rules, while a deterministic Prolog engine performs the actual deduction. The system’s core contributions are:

- A **high-level logical description language** (**EUCLID-IR**) for expressing business rules, security policies, and other domain constraints in a human-readable, declarative form.
- An automatic **translation layer** that compiles these descriptions into executable SWI-Prolog clauses, preserving structure and enabling straightforward auditing.
- A **compact MCP tool interface** that exposes reasoning operations (deduction, diagnosis, scenario analysis, validation) and supports a structured translate-run-inspect-repair loop, including access to proof trees for interpretability.

- Representative **use cases** demonstrating how EUCLID-MCP can be used to enforce business rules and verify security policies in a transparent, verifiable manner.

EUCLID-MCP originated from a practical limitation encountered when applying retrieval-augmented generation (RAG) to business rule enforcement. In this setting, queries must be matched against a corpus of formal or semi-formal rules, and decisions must follow logically from those rules rather than from approximate semantic similarity. Standard RAG pipelines, which rely on vector search and nearest-neighbor retrieval, proved inadequate: they could surface “similar” rules but could not guarantee that the retrieved subset was logically sufficient or consistent, nor could they provide formal proofs of the derived conclusions. In effect, semantic search behaved like “using a drill to hammer a nail”: powerful, but mismatched to the task.

This observation led us to revisit rule-based expert systems and logic programming, where knowledge is encoded as explicit rules and inference is performed by a deterministic engine capable of deriving and explaining conclusions. Building on this foundation, we designed EUCLID-MCP as an MCP server that exposes a Prolog-based reasoning engine to LLM clients. While recent work such as PrologMCP [12] and LogicLease [4] also explores the integration of Prolog with LLMs and MCP, EUCLID-MCP is primarily motivated by the need to overcome the limitations of semantic retrieval in rule-centric applications, and it emphasizes a domain-oriented logical description layer tailored to business and security policies.

The remainder of this paper is organized as follows. Section 2 reviews background and related work on neuro-symbolic AI, logic programming, and MCP. Section 3 describes the EUCLID-MCP architecture, including the logical description language, translation to Prolog, and MCP integration. Section 4 presents use cases in business rule enforcement and security policy verification. Section 5 discusses benefits, limitations, and relations to existing systems. Section 6 concludes.

2 Background and Related Work

This section situates EUCLID-MCP at the intersection of three strands of research and practice: (i) the limitations of retrieval-augmented generation (RAG) for rule-centric tasks, (ii) the tradition of rule-based expert systems and logic programming, and (iii) recent neuro-symbolic approaches that combine LLMs with formal reasoning engines. We also briefly review the Model Context Protocol (MCP) as an emerging standard for tool integration.

2.1 Retrieval-Augmented Generation and Its Limits for Rules

Retrieval-augmented generation (RAG) has become a dominant pattern for grounding LLMs in external knowledge. In a typical RAG pipeline, a user query is embedded into a vector space, similar documents or passages are retrieved via approximate nearest-neighbor search, and the LLM conditions its generation on this retrieved context. This approach works well for open-ended question answering, summarization, and many knowledge-intensive tasks where “semantic similarity” is a good proxy for relevance.

However, RAG based on semantic similarity is fundamentally mismatched to **rule-centric applications**, such as business rule enforcement, policy compliance, and security constraint verification. In these settings:

- Correctness depends on **logical consequence**: a decision must follow from an explicit set of rules and facts, not from textual similarity.
- The retrieved subset of rules must be **logically sufficient** and **consistent** with respect to the query; nearest-neighbor retrieval provides no such guarantees.
- Auditing and explainability require **formal proofs** or at least traceable derivations, not just “the model saw similar rules in the context.”

Recent analyses of RAG in practice highlight these failure modes: semantic retrieval can surface superficially relevant rules while missing critical exceptions, interactions, or negations, leading to inconsistent or unsafe conclusions. In effect, using semantic search to enforce formal rules is akin to “using a drill to hammer a nail”: the tool is powerful, but the underlying operation (approximate matching vs. logical inference) is wrong for the task.

These limitations motivate architectures in which retrieval is complemented or replaced by a **symbolic reasoning layer** that can guarantee logical correctness and provide explicit derivation traces.

2.2 Rule-Based Expert Systems and Logic Programming

Before the current LLM era, **expert systems** were the dominant paradigm for encoding and automating domain expertise, particularly in settings requiring rigorous rule application and explainability. Knowledge was represented as **if-then rules** and facts, and inference engines used forward or backward chaining to derive conclusions and answer queries. A key advantage of this approach was **transparency**: the system could explain *why* a conclusion was reached by exposing the chain of rule applications.

Logic programming, and in particular **Prolog**, became a natural foundation for many expert systems. Prolog’s declarative semantics allow knowledge to be expressed as Horn clauses, while its built-in inference mechanism (backtracking search with unification) automatically derives consequences from facts and rules. Classic work demonstrated Prolog’s suitability for expert systems, emphasizing:

- A clear separation between **knowledge base** (facts and rules) and **inference engine**.
- The ability to generate **explanations** and **proof traces** by inspecting the search process.
- Compact, human-readable encodings of complex domain logic.

Although expert systems and Prolog fell out of mainstream attention during the rise of statistical machine learning and deep learning, they remain conceptually relevant for tasks where **deterministic reasoning**, **auditability**, and **formal guarantees** are essential. Recent discussions on Prolog’s role in modern AI highlight its enduring value as a symbolic backbone for hybrid systems, especially when combined with neural components [8].

2.3 Neuro-Symbolic AI: Combining LLMs and Symbolic Reasoning

Neuro-symbolic AI seeks to integrate the strengths of neural models (pattern recognition, language understanding) with symbolic methods (logical inference, structured knowledge). A common pattern is to use LLMs to parse natural language and generate structured representations, which are then processed by a symbolic engine to ensure correctness and explainability.

Recent work in this direction includes:

- **Neuro-Symbolic Compliance** frameworks that combine LLM-based understanding with SMT-based reasoning to analyze financial regulations in an interpretable and verifiable manner [9].
- Systems that extract policies from natural language using LLMs and enforce them with symbolic engines, explicitly designing for **fail-safe behavior** when the extracted rules cannot fully discharge a decision.
- Approaches that compile rules into logical forms and use satisfiability or constraint solvers to verify properties, often in multi-agent or domain-specific configurations.

Within this landscape, **Prolog-based neuro-symbolic systems** have gained renewed interest. Several lines of work explore:

- Training or prompting LLMs to emit Prolog code that is then executed by an external solver, improving both answer accuracy and the reliability of reasoning proofs [3].
- Using Prolog derivation logs to generate **causal and human-readable explanations**, addressing the “black-box” nature of pure neural reasoning.
- Comparing adaptive neuro-symbolic systems with traditional, hard-coded business rule engines, particularly in compliance-critical domains such as finance and healthcare.
- Applying neuro-symbolic methods to enterprise systems (e.g., CRM, ERP) where business logic must be both flexible and formally analyzable.

Prior work has approached LLM hallucination largely as a **post-hoc mitigation** problem, using multi-agent review pipelines and dedicated scoring metrics to detect and correct unsupported claims after generation [11]. More recent architectures combine agentic review with Nested Learning and semantic caching to reduce hallucination while lowering computational cost [10]. EUCLID-MCP takes a complementary, **architectural** route: rather than correcting fabrications after they occur, it removes the reasoning step from the model entirely, delegating deduction to a deterministic engine.

2.4 Model Context Protocol (MCP)

The **Model Context Protocol (MCP)** is an emerging open standard for connecting LLM applications to external tools, data sources, and services [5]. MCP defines a uniform interface through which LLM clients can:

- Discover available **tools** and **resources** (e.g., databases, APIs, reasoning engines).
- Invoke tools with structured inputs and receive structured outputs.
- Compose multiple tools into complex workflows while maintaining a consistent interaction model.

MCP has quickly attracted implementations from major organizations and a growing ecosystem of community-maintained servers, ranging from filesystem and code access to specialized data processors. However, until

recently, few MCP servers have exposed **formal reasoning engines** as first-class components. Early efforts such as PrologMCP [12] demonstrate the feasibility of exposing Prolog as an MCP tool, primarily targeting generic logical reasoning tasks.

EUCLID-MCP builds on the MCP abstraction but specializes it for **policy and rule modeling**, offering:

- A domain-oriented logical description language tailored to business rules and security constraints.
- Integrated translation to Prolog, safety checks, and error reporting.
- Tools explicitly designed for **policy evaluation**, **diagnostic analysis**, **scenario testing**, and **KB validation**, rather than generic Prolog execution.

In doing so, EUCLID-MCP contributes to the emerging class of MCP-based neuro-symbolic systems, with a clear focus on overcoming the limitations of semantic retrieval in rule-centric applications.

3 Euclid-MCP Architecture

This section describes the design of EUCLID-MCP, from high-level architecture to the EUCLID-IR language, translation to Prolog, and MCP integration. The system is implemented as a Python-based MCP server that invokes SWI-Prolog as a local reasoning backend and exposes a small, purpose-built tool surface for policy and rule reasoning.

The use of **SWI-Prolog** in the current prototype is a **tactical choice**, not an architectural dependency. EUCLID-MCP is designed around an **engine-agnostic intermediate representation** (EUCLID-IR) that can be lowered to Prolog or to alternative inference engines (e.g., Datalog engines, SMT solvers, custom rule engines) without changing the user-facing APIs or the MCP tool surface.

3.1 High-Level Architecture

EUCLID-MCP follows a **hybrid cognitive architecture** in which an LLM describes the world in terms of facts and rules, while a deterministic inference engine performs the actual deduction. The main components are:

LLM client Any MCP-capable LLM application (e.g., an AI assistant, agent framework, or custom tooling). The LLM is responsible for interpreting natural language inputs from users, generating structured descriptions of facts and rules in EUCLID-IR, invoking EUCLID-MCP tools to evaluate queries and inspect derivations, and rendering results and explanations back to the user.

Euclid-MCP server A Python process implementing the MCP server interface via **FastMCP**. It provides a EUCLID-IR parser that validates and normalizes facts, rules, and queries; an intermediate representation (EUCLID-IR) that is independent of any specific inference engine; a lowering layer that compiles EUCLID-IR into executable SWI-Prolog code; safety and sanitization to restrict the set of allowed Prolog constructs; and a tool interface for query evaluation, diagnostic analysis, scenario testing, and KB validation.

Inference backend In the current prototype, **SWI-Prolog** acts as the deterministic inference engine, invoked as an external process. The server translates EUCLID-IR into a self-contained Prolog program, writes it to a temporary file and executes it via `swipl` subprocess, parses structured JSON output containing solutions and proof trees, and returns results to the MCP client as typed data structures.

This separation of concerns ensures that the LLM never needs to perform multi-step logical reasoning internally; it only needs to describe the problem and interpret the results. All deduction is delegated to the inference backend, which guarantees logical correctness with respect to the encoded rules and facts.

3.2 Euclid-IR: An Engine-Agnostic Intermediate Representation

The core abstraction in EUCLID-MCP is **EUCLID-IR**, a declarative language for logical inference designed to be:

- **Readable** — Syntax optimized for both humans and LLMs.
- **Minimal** — Only the essential constructs of Horn-clause logic.
- **Deterministic** — Every query has a finite, traceable proof.
- **Backend-agnostic** — EUCLID-IR is an intermediate layer; today it targets Prolog, tomorrow it could target other engines.

A EUCLID-IR knowledge base consists of:

- **Facts** – ground assertions about the world.
- **Rules** – implications of the form *head IF body*.
- **Queries** – goals to prove, prefixed with ?.
- Optional **version directive** (@version 1.0) and **comments** (# or //).

3.2.1 Core Constructs

Facts. Ground atoms that describe the current state of the world:

```
parent(tom, bob)
color(apple, red)
active(user_42)
rainy
```

Predicate names are lowercase identifiers; arguments are atoms, integers, or variables. Zero-arity facts (e.g., **rainy**) are allowed.

Variables. Unknown or generic values, prefixed with \$:

```
$x
$who
$user_name
```

Variables must start with \$ followed by a lowercase letter; they are case-sensitive and translated to Prolog variables (capitalized) during lowering.

Rules. Logical implications with a head and a body:

```
mortal($x) IF human($x)
ancestor($x, $y) IF parent($x, $y)
ancestor($x, $y) IF parent($x, $z) AND ancestor($z, $y)
```

The body is a conjunction of conditions connected by AND. Rules can span multiple lines for readability, with continuation implied when a line ends in IF or AND.

Negation. Closed-world negation using NOT:

```
blocked($user) IF NOT active($user)
eligible($user) IF registered($user) AND NOT blocked($user)
```

NOT p succeeds when p cannot be proven, corresponding to Prolog's \+.

Queries. Goals to prove, prefixed with ?:

```
? mortal(socrates)
? ancestor(tom, $who)
? can_access($user, $res) AND resource($res, _, _, _, secret)
```

Queries may include variables (to find bindings) or be ground (boolean checks). Conjunctions with AND are allowed.

Arithmetic. Numeric comparisons and evaluations in rule bodies:

```
stale($user) IF user($user) AND last_login($user, $days) AND $days >
90
adult($person) IF age($person, $age) AND $age >= 18
```

Supported operators include `>`, `>=`, `<`, `<=`, `:=`, `=\=`, and `is`, which are passed through to the backend (currently Prolog) and evaluated at deduction time.

Wildcards. The `_` symbol represents an anonymous variable:

```
resource(apple, $color, _, _, _, _)
```

Comments. Two styles are supported:

```
# This is a comment
// This is also a comment

parent(tom, bob) # inline comment
```

Version directive. An optional first line can declare the EUCLID-IR version:

```
@version 1.0

parent(tom, bob)
? parent($x, $y)
```

If omitted, version 1.0 is assumed. Future versions are intended to be backward compatible.

3.2.2 Design Choices and Limitations

EUCLID-IR targets **Horn-clause logic** — the core of Prolog without advanced features:

- **Supported:** facts, rules, negation (closed-world), arithmetic, multi-line rules, conjunction queries, wildcards.
- **Not supported:** disjunction (OR), cut (!), list syntax, `findall`/`bagof`, dynamic assert/retract, modules, strings (only atoms).

These limitations are intentional: they keep the language minimal, deterministic, and easy to lower to multiple backends. Workarounds exist for common patterns (e.g., encoding disjunction as multiple rules, representing lists via indexed facts).

EUCLID-IR also supports a **YAML format** for structured input, though the text format is recommended for conciseness and readability.

3.3 Translation to Prolog

The Prolog lowering layer converts EUCLID-IR into executable SWI-Prolog code. This process is fully automated and deterministic:

1. **Mapping to Clauses.** Each EUCLID-IR fact is translated directly into a Prolog fact. Each EUCLID-IR rule is translated into a Prolog clause. For example:

```
mortal(X) :- human(X).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Queries are encoded as top-level goals that can be invoked from the MCP tools.

2. **Variable Translation.** EUCLID-IR variables (e.g., `$x`, `$who`) are mapped to Prolog variables by capitalizing the name (e.g., `X`, `Who`). This isolates users and the LLM from Prolog’s syntactic idiosyncrasies (uppercase variables).
3. **Keyword and Operator Mapping.** EUCLID-IR keywords and operators are translated to their Prolog equivalents: `IF` $\rightarrow$ `:-`, `AND` $\rightarrow$ `,`, `NOT` $\rightarrow$ `\+`, `?` $\rightarrow$ `?-`. Arithmetic operators (`>`, `>=`, etc.) are passed through verbatim.
4. **Safety and Sanitization.** The translation layer enforces safety properties: a sanitizer validates that only allowed predicate constructors appear in the input; potentially dangerous built-ins (e.g., file I/O, network access) are excluded from the generated code; input size is capped (500 KB) and execution is bounded by timeout (30 s by default).

This design ensures that the generated Prolog code is both **auditable** (humans can read and verify the mapping) and **safe** (the LLM cannot arbitrarily execute system commands via Prolog). Future backends will follow the same pattern: EUCLID-IR $\rightarrow$ target language, with analogous safety controls.

3.4 MCP Integration: Tools and Interaction Pattern

EUCLID-MCP exposes its functionality through a small, well-defined set of four MCP **tools**, following established MCP design patterns. Each tool is a stateless function that accepts a knowledge base as input and returns a typed result. The tools are:

reason Purpose: Main deduction — evaluate a knowledge base and return solutions with proof trees.

Input: `knowledge` (EUCLID-IR text or YAML), optional `query` override, `max_solutions` (default 5), `max_depth` (default 30).

Output: A list of solutions, each containing variable bindings (`substitutions`) and a proof tree with nodes of type `fact`, `rule`, or `and`.

Example: Given facts about parents and rules about ancestry, `reason` returns all solutions for `ancestor(tom, $who)` with full derivation chains.

diagnose Purpose: Query analysis — understand why a query succeeds or fails.

Input: `knowledge`, `query`, `mode` (one of `why`, `why_not`, `what_needs`), `max_solutions`, `max_depth`.

Output: A `DiagnosisResult` with `holds` (boolean), `findings[]` (list of issues detected), `conclusion` (human-readable summary), and optionally a `proof`.

Modes: `why` — explain why a query holds (or that it doesn't); `why_not` — explain why a query fails (missing facts/rules); `what_needs` — suggest what would make a false query true.

Example: `diagnose(knowledge, "mortal(plato)", mode="why_not")` returns findings like “No facts or rules defined for ‘mortal’” and suggests what to add.

what_if Purpose: Scenario analysis — apply modifications to a knowledge base and compare results before and after.

Input: `base_knowledge`, `modifications` (lines prefixed with + to add or - to remove facts), `query`, `max_solutions`, `max_depth`.

Output: A `WhatIfResult` with `before_count`, `after_count`, `delta`, `solutions_before`, `solutions_after`, and a `conclusion` describing the impact.

Example: `what_if(base_kb, "+ human(plato)", "mortal($who)")` shows how adding a fact changes the number of solutions.

check_kb Purpose: Knowledge base validator — check for syntax errors, undefined predicates, circular rules, and duplicates before running deduction.

Input: knowledge (EUCLID-IR text or YAML).

Output: A `KBCheckResult` with `valid` (boolean), `errors[]`, `warnings[]`, `facts_count`, `rules_count`, `predicates_count`.

Example: `check_kb(knowledge)` returns `{valid: true, errors: [], warnings: [], facts_count: 2, rules_count: 1}`.

These tools are designed to support a **translate-run-inspect-repair loop**:

1. **Validate:** The LLM calls `check_kb` to verify the knowledge base is well-formed before reasoning.
2. **Translate:** The LLM generates a EUCLID-IR knowledge base (facts + rules + query).
3. **Run:** The LLM invokes `reason` to obtain answers with proof trees.
4. **Inspect:** If the result is unexpected, the LLM calls `diagnose` to understand why a query succeeds or fails.
5. **Repair:** Based on the diagnosis, the LLM may refine the knowledge base (e.g., add missing rules, correct mis-encoded conditions) and re-run the query.
6. **Explore:** The LLM uses `what_if` to test hypothetical modifications before applying them.

Errors from the inference backend (e.g., syntax errors, unsatisfiable queries) are returned as **readable text messages**, consistent with MCP best practices, so the LLM can use them as feedback for self-correction.

3.5 Implementation Details

The current prototype of EUCLID-MCP is implemented in **Python** and uses SWI-Prolog as the reasoning backend. Key implementation choices include:

Prolog Integration The server invokes SWI-Prolog as an external subprocess. For each reasoning call: the generated Prolog code is written to a temporary `.pl` file; the `swipl` binary is invoked with `subprocess.run()` and a configurable timeout (default 30s); structured JSON output (solutions + proof trees) is captured from stdout and parsed; the temporary file is cleaned up after execution. This approach prioritizes **simplicity and portability**: it requires no compiled extensions, no foreign language interfaces, and no persistent Prolog process.

MCP Transport EUCLID-MCP supports two transport modes: **stdio** — the default MCP transport, used when the server is launched by a local client (e.g., Claude Desktop, OpenCode, Cursor); **HTTP API** — a standalone REST server (via `integrations/euclid_api.py`) exposing POST endpoints at `/reason`, `/diagnose`, `/what-if`, `/check-kb`, plus a GET `/health` endpoint. This mode is designed for integration with automation platforms (n8n, Zapier, Make) and remote access.

Packaging and Distribution The server is distributed as a Python package via PyPI (`pip install euclid-mcp`) and can also be built as a Docker image that bundles SWI-Prolog, eliminating the need for a local Prolog installation.

This implementation provides a practical, reusable foundation for integrating deterministic logical reasoning into LLM-based applications, with a particular focus on business rules and security policies. The presence of EUCLID-IR ensures that the system can evolve beyond Prolog as new inference backends become desirable or necessary.

4 Use Case: IT Security & Compliance

To demonstrate the practical applicability of EUCLID-MCP, we developed a realistic security compliance model for a cloud-based organization. This use case illustrates how EUCLID-IR can encode multi-layer policies, support diverse reasoning patterns, and enable explanation and counterfactual analysis that are beyond the reach of semantic retrieval alone.

The full knowledge base, along with all queries and scenarios described in this section, is available in the public repository at https://github.com/meob/Euclid-MCP/tree/main/examples/07_it_security_compliance.

4.1 Scenario Description

The modeled organization operates a multi-team engineering and operations structure in a cloud environment (AWS-flavored). Compliance requirements are derived from:

- **External benchmarks:** CIS Amazon Web Services Foundations Benchmark, with controls covering account management, S3, CloudTrail, networking, GuardDuty, RDS, EC2, Lambda, and more.

- **Internal policies:** IAM best practices, role hierarchies, environment tiers, data classification, approval workflows, and incident response procedures.

The knowledge base is organized into three conceptual layers:

1. **Standards (Layer 1):** CIS controls with severity levels (critical, high, medium, low). IAM best-practice rules encoding risks such as separation-of-duties violations, excessive permissions, stale access, privilege escalation, root account violations, and service account risks.
2. **Policies (Layer 2):** Role hierarchy (intern $\rightarrow$ junior_dev $\rightarrow$ mid_senior_dev $\rightarrow$ senior_dev $\rightarrow$ tech_lead $\rightarrow$ eng_manager $\rightarrow$ director $\rightarrow$ vp_engineering $\rightarrow$ cto, plus operations, security, and product roles). Environment tiers (production, golden, staging, development, sandbox) with associated deployment level requirements, encryption/backup requirements, and audit log requirements. Data classification (public, internal, confidential, secret) and role clearance levels. Access control rules, critical operations, approval workflows, MFA requirements, and incident-mode restrictions.
3. **Data (Layer 3):** 30 users (in the small version) with attributes such as department, role(s), permissions, MFA status, last login time, account type (human/service), console access, and access keys. 50 resources (EC2, S3, RDS, KMS, Lambda, ECS, EKS, SNS, SQS, DynamoDB) with attributes such as environment, encryption status, backup status, access level (public/private), and data classification. CIS control applicability facts linking specific resources to relevant CIS controls.

A larger variant of the same model (200 users, 300 resources, $\sim 3,872$ facts) is used for performance evaluation and stress testing.

4.2 Representative Queries

We defined a set of 10 canonical queries covering a wide range of reasoning patterns: single-hop permission checks, multi-hop policy reasoning, temporal and threshold patterns, cross-policy violations, and resource audits. Below we present four representative queries that illustrate the diversity of questions EUCLID-MCP can answer.

4.2.1 Permission Check Through Role Hierarchy (Q1)

Question: “Can user_0005 manage servers?”

```
? user_has_permission(user_0005, manage_servers)
```

This query checks whether a specific user has a specific permission via the role hierarchy. The `user_has_permission/2` predicate is derived from the user’s assigned role(s), and the `role_has_permission/2` predicate, which accounts for direct role permissions and inherited permissions via the `inherits/2` relation.

Result: The query succeeds. The proof tree shows that `user_0005` has the `sysadmin` role, which directly grants `manage_servers` permission.

4.2.2 Multi-Hop Policy Reasoning: Deployment to Production (Q2)

Question: “Which roles can deploy code to production?”

```
? can_deploy($who, production)
```

The `can_deploy/2` predicate encodes a composite policy: the user must have the `deploy_code` permission; the user’s role level must meet or exceed the requirement for the target environment (production requires level ≥ 6); the role hierarchy and `deploy_role_level/2` facts determine each role’s level.

Result: The query returns bindings for `$who` corresponding to users with roles at level 6 or higher (e.g., `director`, `vp_engineering`, `cto`) who also have `deploy_code` permission.

4.2.3 Cross-Policy Violation: Separation of Duties (Q6)

Question: “Which users violate separation of duties?”

```
? violates_separation_of_duties($who)
```

The `violates_separation_of_duties/1` predicate encodes two conflict patterns: having both `deploy_code` and `approve_deploy` permissions; having both `create_role` and `assign_role` permissions.

Result: In the small KB, the query yields 2 users; in the larger KB, it yields 11. For each violating user, `diagnose` can generate a human-readable explanation listing the conflicting permissions and the rules that classify them as a violation.

4.2.4 Resource Audit: Unencrypted Production Resources (Q7)

Question: “Which production resources are not encrypted?”


```
? resource($name, production, not_encrypted, _, _, _)
```

This query performs a simple pattern match over resource facts, filtering by environment (`production`) and encryption status (`not_encrypted`).

Result: The query returns a list of resource names (e.g., specific EC2 instances, S3 buckets, RDS databases) that are in production but not encrypted. This directly supports CIS compliance audits (e.g., CIS 2.7 for RDS encryption, CIS 7.1 for EC2).

4.3 Additional Query Patterns

The full query set includes several other patterns that further demonstrate the expressivity of EUCLID-IR:

- **Clearance-based access (Q3):** “Which users can access secret data?”

```
? can_access_resource($who, $res) AND resource($res, _, _, _, _, secret)
```

This query joins user clearance levels with resource classification, enforcing that `user_max_clearance` $\geq$ `resource_classification_level`.

- **Negative query (Q8):** “Can an intern write code?”

```
? user_has_permission($who, write_code) AND has_role($who, intern)
```

Expected result: empty set, since interns only have `read_code`. This validates that the policy correctly restricts interns.

- **Temporal / IAM hygiene (Q5, Q9):** “Which users have stale access (over 90 days)?”

```
? stale_access($who)
```

“Which users have excessive permissions (more than 15)?”

```
? excessive_permissions($who, $count)
```

These encode AWS IAM best-practice patterns for detecting stale credentials and least-privilege violations.

Together, these queries cover single-hop, multi-hop, temporal, cross-policy, threshold, and resource-audit reasoning patterns.

4.4 Explanations and Diagnostic Reasoning

Beyond answering queries, EUCLID-MCP can explain *why* a conclusion holds or fails. We defined three diagnostic questions to illustrate this capability.

4.4.1 Why Does a User Have a Permission? (Q11)

Question: “Why does eng_0008 have `manage_servers` permission?”

Using the `diagnose` tool with `mode="why"`:

```
{"knowledge": "...",  
  "query": "user_has_permission(eng_0008, manage_servers)",  
  "mode": "why"}
```

Explanation: The system generates a proof tree showing that `eng_0008` has the `sysadmin` role, which has the `manage_servers` permission via `role_permission(sysadmin, manage_servers)`.

4.4.2 Why Does a User Lack a Permission? (Q12)

Question: “Why doesn’t eng_0002 (intern) have `deploy_code` permission?”

Using the `diagnose` tool with `mode="why_not"`:

```
{"knowledge": "...",  
  "query": "user_has_permission(eng_0002, deploy_code)",  
  "mode": "why_not"}
```

Explanation: The system reports that `eng_0002` has the `intern` role, which only has `read_code` and `run_tests` permissions. There is no inheritance path from `intern` to any role with `deploy_code`.

4.4.3 What Is Needed to Gain a Permission? (Q13)

Question: “What would eng_0002 need to deploy code?”

Using the `diagnose` tool with `mode="what_needs"`:

```
{"knowledge": "...",  
  "query": "user_has_permission(eng_0002, deploy_code)",  
  "mode": "what_needs"}
```

Explanation: The system identifies missing conditions: `eng_0002` would need a role that has `deploy_code` permission (e.g., `senior_dev`, `tech_lead`, or higher).

4.5 What-If Analysis and Counterfactuals

EUCLID-MCP can also evaluate hypothetical changes to the knowledge base, supporting impact analysis and policy design. We defined three what-if scenarios using the `what_if` tool:

4.5.1 Role Promotion (W1)

Question: “What if `eng_0002` (intern) is promoted to `senior_dev`?”

```
{
  "base_knowledge": "...",
  "modifications": "- has_role(eng_0002, intern)\n+ has_role(eng_0002, senior_dev)",
  "query": "user_has_permission(eng_0002, deploy_code)"
}
```

Result: In the base configuration, the query fails (interns cannot deploy). Under the modified configuration, the query succeeds, and the explanation shows that `senior_dev` has `deploy_code` permission, which is inherited by `eng_0002`.

4.5.2 Adding a Compliant Resource (W2)

Question: “What if a new encrypted production database is added?”

```
{
  "base_knowledge": "...",
  "modifications": "+ resource(new_db, production, encrypted, db_team, 2026-07-01, database)\n+ resource_type(new_db, database)",
  "query": "resource($name, production, encrypted, _, _, _) AND resource_type($name, database)"
}
```

Result: The query succeeds under the modified configuration, confirming that the new resource complies with encryption and environment requirements.

4.5.3 Role Addition (W3)

Question: “What if `ops_0001` (helpdesk) gets the `sysadmin` role?”

```
{
  "base_knowledge": "...",
  "modifications": "+ has_role(ops_0001, sysadmin)",
  "query": "user_has_permission(ops_0001, manage_servers)"
}
```

Result: In the base configuration, `ops_0001` (helpdesk) does not have `manage_servers`. Under the modified configuration, the query succeeds, and the explanation shows the new permission path.

4.6 Discussion

This use case demonstrates several key points:

1. **Expressivity:** EUCLID-IR can encode realistic, multi-layer security and compliance policies, including role hierarchies, environment tiers, data classification, approval workflows, and CIS control applicability.
2. **Diverse reasoning patterns:** The query set covers single-hop, multi-hop, temporal, cross-policy, threshold, and resource-audit patterns, as well as diagnostic and counterfactual reasoning.
3. **Explainability:** For each conclusion, EUCLID-MCP can generate a proof tree and render it in human-readable form, supporting audits, onboarding, and policy design.
4. **Determinism and auditability:** Unlike semantic RAG, which can only retrieve “similar” policies, EUCLID-MCP provides deterministic, auditable answers to questions such as “Is this user compliant?” or “Which resources violate CIS controls?”
5. **Scalability:** The same model scales from a small KB (~ 30 users, ~ 50 resources) to a larger KB (~ 200 users, ~ 300 resources) without changing the policy rules, demonstrating that EUCLID-IR can support realistic organizational scales.

4.7 Summary of Queries

For reference, Table 1 summarizes all canonical queries, diagnostic questions, and what-if scenarios.

5 Evaluation

We evaluate EUCLID-MCP along two dimensions:

1. **Qualitative expressivity:** Can EUCLID-IR encode realistic, multi-layer policies and support diverse reasoning patterns (as demonstrated in Section 4)?

Table 1: Summary of queries and scenarios in the IT security & compliance use case.

ID	Type	Question	Pattern
Q1	Query	Can user_0005 manage servers?	Single-hop permission check
Q2	Query	Which roles can deploy code to production?	Multi-hop policy reasoning
Q3	Query	Which users can access secret data?	Clearance vs classification
Q4	Query	Can a tech_lead deploy to golden environment?	Multi-role deployment
Q5	Query	Which users have stale access (over 90 days)?	Temporal / IAM hygiene
Q6	Query	Which users violate separation of duties?	Cross-policy violation
Q7	Query	Which production resources are not encrypted?	Resource audit
Q8	Query	Can an intern write code?	Negative query (expected e
Q9	Query	Which users have excessive permissions (>15)?	Threshold / least privilege
Q10	Query	Which S3 buckets in production are not encrypted?	Combined filter
Q11	Diagnose	Why does eng_0008 have manage_servers permission?	Why-explanation
Q12	Diagnose	Why doesn't eng_0002 (intern) have deploy_code permission?	Why-not explanation
Q13	Diagnose	What would eng_0002 need to deploy code?	What-needs analysis
W1	What-if	What if eng_0002 (intern) is promoted to senior_dev?	Role promotion
W2	What-if	What if a new encrypted production database is added?	Resource addition
W3	What-if	What if ops_0001 (helpdesk) gets the sysadmin role?	Role addition

2. **Quantitative performance and accuracy:** How does the system scale with knowledge base size, and how does it compare to LLM-only reasoning on logical tasks?

The evaluation focuses on the IT security & compliance use case from Section 4, using both the small (~ 30 users, ~ 50 resources, ~ 578 facts) and large (~ 200 users, ~ 300 resources, $\sim 3,872$ facts) knowledge bases, plus a synthetic RBAC benchmark at 1,000 users.

5.1 Expressivity

The security compliance model demonstrates that EUCLID-IR can encode:

- **Multi-layer policies:** External standards (CIS controls), internal policies (role hierarchies, environment tiers, data classification), and concrete data (users, resources, configurations).
- **Diverse reasoning patterns:** Single-hop permission checks, multi-hop policy reasoning, temporal patterns (stale access), threshold patterns (excessive permissions), cross-policy violations (separation of duties), and resource audits (unencrypted production resources).

- **Diagnostic and counterfactual reasoning:** Why/why-not explanations, what-needs analysis, and what-if scenarios for role engineering and infrastructure planning.

These capabilities go well beyond what semantic RAG can provide. While RAG can retrieve “similar” policies or past incidents, it cannot deterministically derive whether a user violates separation of duties, which resources fail CIS controls, or what would change if a role were modified.

5.2 Reasoning Benchmarks

To quantify the benefits of EUCLID-MCP over LLM-only reasoning, we conducted two benchmarks comparing:

- **A: llama3.1:8b** (small local LLM).
- **B: qwen3-coder:480b-cloud** (large cloud LLM).
- **C: llama3.1:8b + Euclid-MCP** (small LLM augmented with deterministic inference).

5.2.1 Small-Scale Reasoning Benchmark

Task: 5 logical reasoning tasks (genealogy, taxonomy, RBAC) with 5–15 facts each. **Goal:** Assess whether LLMs alone can handle small, self-contained logical problems.

Table 2: Small-scale reasoning benchmark results.

Q	Task	GT	A (8B)	B (480B)	C (8B+Euclid)
Q1	Genealogy (deep chain)	Yes	✓	✓	✓
Q2	Taxonomy (property inheritance)	Yes	✓	✓	✓
Q3	Taxonomy (negative inference)	No	✓	✓	✓
Q4	RBAC (permission inheritance)	Yes	✓	✓	✓
Q5	RBAC (negative)	No	✓	✓	✓
Accuracy			5/5	5/5	5/5
Avg time			4 772 ms	2 180 ms	2 542 ms
Avg tokens (in / out)			131 / 118	130 / 133	254 / 46

Conclusion: With small KBs (5–50 facts), all three conditions achieve equivalent accuracy. EUCLID-MCP matches LLM accuracy, but at slightly higher input tokens (254 vs. 131) due to the EUCLID-IR encoding. Execution time is comparable (2.5 s vs. 4.8 s / 2.2 s).

5.2.2 Large-Scale RBAC Benchmark

Task: 1,000 synthetic users, 7 roles with hierarchy, 17 base permissions, 20 direct grants — 1,053 facts total. **Goal:** Assess performance at a scale where LLM working memory fails.

Table 3: Large-scale RBAC benchmark results.

Q	Task	GT	A (8B)	B (480B)	C (8B+Euclid)
Q1	Count users with <code>delete_repo</code>	31	× 1	× 1	✓ 31
Q2	Can user_0142 <code>push_code</code> ?	Yes	✓	✓	✓
Q3	Count users with <code>deploy</code>	103	× 100	× 901	✓ 103
Q4	Can user_0834 <code>read_logs</code> ?	Yes	× No	✓	✓
Q5	Can user_0222 <code>manage_billing</code> ? (direct)	Yes	✓	× No	✓
Accuracy			2/5	2/5	5/5
Avg time			6 966 ms	3 695 ms	963 ms
Avg tokens (in / out)			386 / 165	435 / 212	421 / 12

Conclusion: At scale (1,000+ facts), LLMs alone hallucinate systematically — both 8B and 480B cloud give wrong counts and miss explicit facts. EUCLID-MCP delivers exact answers every time, while being **faster** (963 ms vs. 6,966 ms) and **more token-efficient** (12 vs. 165 output tokens) because the LLM generates only a simple query instead of fallacious reasoning.

5.3 Summary of Benchmark Findings

Table 4: Benchmark summary.

KB size	LLM alone	LLM + Euclid-MCP
Small (5–50 facts)	✓ Sufficient accuracy	△ Comparable accuracy, higher input tokens
Large (1,000+ facts)	× Hallucinates systematically	✓ Exact deduction, faster, more token-efficient

Key takeaway: EUCLID-MCP proves its value at scale. When facts fit in an LLM’s context window and the reasoning chain is shallow, the overhead of a deterministic engine is rarely justified. When the KB exceeds a few hundred facts — or when exact counting, intersection, or multi-hop inheritance is required — EUCLID-MCP delivers exact answers while LLMs of any size hallucinate systematically.

5.4 Comparison with Semantic RAG

To contextualize these results, consider the typical workflow for answering a compliance question using semantic RAG:

1. **Retrieve** relevant policy documents or past incidents based on the query.
2. **Read** the retrieved documents and attempt to infer the answer.
3. **Guess** or approximate the conclusion, often with significant uncertainty.

In contrast, EUCLID-MCP:

1. **Encodes** policies as formal rules in EUCLID-IR.
2. **Derives** answers deterministically using logical inference.
3. **Explains** the derivation with proof trees.

Table 5: Comparison: Semantic RAG vs. EUCLID-MCP.

Aspect	Semantic RAG	Euclid-MCP
Reasoning	Approximate, based on similarity	Deterministic, based on formal rules
Auditability	Low (retrieved snippets, no proofs)	High (proof trees, rule references)
Policy enforcement	Not possible (retrieval only)	Direct (rules encode enforcement logic)
Counterfactuals	Not supported	Supported (what-if scenarios)
Accuracy at scale	Degrades with KB size	Exact, independent of KB size
Latency	Depends on retrieval + LLM inference	Typically <1s for tested queries

This comparison underscores that EUCLID-MCP is not a replacement for RAG, but rather a complementary tool for scenarios requiring deterministic reasoning and auditability.

5.5 Limitations

While the evaluation demonstrates strong expressivity and accuracy, several limitations should be noted:

1. **Horn-clause restriction:** EUCLID-IR does not support disjunction, cut, list pattern matching, or advanced Prolog features. This limits expressivity for certain problem domains (e.g., complex data structures, non-monotonic reasoning beyond negation as failure).
2. **Scalability beyond 10K facts:** While the system performs well for KBs up to $\sim 4,000$ facts, larger KBs (e.g., 100,000+ facts) may require optimization (e.g., indexing, tabling, or migration to a Datalog engine).
3. **Backend dependency:** The current prototype relies on SWI-Prolog. While EUCLID-IR is designed to be backend-agnostic, alternative backends (e.g., Datalog, SMT) have not yet been implemented or evaluated.
4. **Integration overhead:** Integrating EUCLID-MCP into an LLM-based application requires additional tooling (e.g., EUCLID-IR generation, error handling, explanation rendering), which may increase development complexity compared to pure RAG.

5.6 Summary

The evaluation demonstrates that EUCLID-MCP:

- **Expresses** realistic, multi-layer policies in EUCLID-IR.
- **Supports** diverse reasoning patterns, including diagnostic and counterfactual analysis.
- **Delivers exact answers** at scale, where LLMs alone hallucinate systematically.
- **Performs well** for interactive use, with median query latencies under 1 s for KBs up to $\sim 1,000$ facts.
- **Complements** semantic RAG by providing deterministic, auditable reasoning for policy enforcement and compliance scenarios.

Future work will explore alternative backends (e.g., Datalog, SMT) to improve scalability and expressivity, as well as integration patterns for production LLM-based applications.

6 Conclusion

This paper presented EUCLID-MCP, a system that augments large language models with deterministic logical reasoning capabilities via the Model Con-

text Protocol. By introducing EUCLID-IR— an engine-agnostic intermediate representation for Horn-clause logic — EUCLID-MCP enables LLMs to encode, evaluate, and explain complex policies and rules without requiring expertise in logic programming or Prolog syntax.

The evaluation demonstrates three key findings:

1. **Expressivity:** EUCLID-IR can encode realistic, multi-layer security and compliance policies, including role hierarchies, environment tiers, data classification, approval workflows, and CIS control applicability. The system supports diverse reasoning patterns (single-hop, multi-hop, temporal, cross-policy, threshold, and resource-audit queries) as well as diagnostic and counterfactual analysis.
2. **Accuracy at scale:** On small knowledge bases (5–50 facts), LLMs alone achieve comparable accuracy to EUCLID-MCP. However, at scale (1,000+ facts), LLMs of any size — from 8B local models to 480B cloud models — hallucinate systematically on counting, intersection, and inheritance queries. EUCLID-MCP delivers exact answers in all cases, while also being faster and more token-efficient.
3. **Explainability and auditability:** For every conclusion, EUCLID-MCP can generate a proof tree and render it in human-readable form. This supports audits, onboarding, role engineering, and policy design — capabilities that are absent from semantic RAG and pure LLM-based reasoning.

The central thesis of this work is that **semantic RAG is the wrong tool for rule enforcement**. RAG excels at retrieving relevant documents, but it cannot deterministically derive whether a user violates separation of duties, which resources fail compliance controls, or what would change if a policy were modified. EUCLID-MCP fills this gap by providing a deterministic inference layer that complements RAG and LLM reasoning.

Practical Implications

For practitioners building LLM-based applications, the implications are clear:

- **Use RAG** for open-ended queries, document retrieval, and knowledge exploration.
- **Use Euclid-MCP** (or similar deterministic reasoning tools) for policy enforcement, compliance checking, access review, and any scenario requiring exact, auditable answers.

- **Combine both:** Use RAG to retrieve relevant policies, then use EUCLID-MCP to evaluate whether a specific configuration or action complies with those policies.

This hybrid approach leverages the strengths of both paradigms: the flexibility and breadth of semantic retrieval, and the precision and auditability of logical inference.

Euclid-MCP as a Stable Reasoning Substrate for Agents

An emerging trend for complex tasks is to use **LLM agents that generate and execute programs**: they collect information, write code, orchestrate tools, and finally return a result. This mitigates hallucination for *procedural* tasks, but it introduces a different problem for **compliance and policy reasoning**:

- Each new compliance question can lead to **new agents** and **new generated programs**.
- Different agents may encode **slightly different interpretations** of the same policy.
- Over time, the organization accumulates **inconsistent, ad-hoc policy implementations** that are hard to audit, compare, or evolve.

EUCLID-MCP addresses this by **externalizing rule knowledge** into a **single, canonical representation** (EUCLID-IR) that is:

- **Shared:** All agents and tools query the same policy definitions.
- **Stable:** Policies change only when the EUCLID-IR knowledge base is updated, not when a new agent is deployed.
- **Auditable:** Every decision can be traced back to specific rules and facts, regardless of which agent initiated the query.
- **Reusable:** The same knowledge base supports access review, deployment approval, incident response, and compliance reporting without re-encoding logic.

In this view, EUCLID-MCP is not an alternative to agents; it is a **shared policy engine** that agents (and RAG systems) call via MCP when they need deterministic answers to rule-based questions.

Broader Impact

EUCLID-MCP represents a step toward **hybrid cognitive architectures** that combine the generality of LLMs with the precision of symbolic reasoning. By exposing logical reasoning as an MCP tool, we lower the barrier to entry for developers who want to integrate deterministic inference into their applications without becoming logic programming experts.

The design of EUCLID-IR—human-readable, backend-agnostic, and easy to translate into multiple target languages—ensures that the system can evolve beyond Prolog as new inference engines become available. This positions EUCLID-MCP as a flexible foundation for future research on hybrid reasoning systems.

Final Thoughts

The results of this evaluation support a simple conclusion: **when facts fit in an LLM’s context window and the reasoning chain is shallow, LLMs alone are sufficient. When they don’t—above a few hundred facts—deterministic reasoning is essential.**

EUCLID-MCP proves its value at scale, delivering exact answers while LLMs of any size hallucinate systematically. By bridging the gap between natural language and formal logic, EUCLID-MCP enables a new class of LLM-based applications that are not only fluent and flexible, but also precise, auditable, and trustworthy.

Availability: The EUCLID-MCP implementation, EUCLID-IR specification, and all benchmarks described in this paper are available at <https://github.com/meob/Euclid-MCP>.

References

- [1] meob. Euclid-MCP: MCP Server for Logical Reasoning — Turns Facts into Formal Proofs. <https://github.com/meob/Euclid-MCP>, 2025. Accessed: 2025-07-22.
- [2] Lauren Nicole DeLong, Ramon Fernández Mir, and Jacques D. Fleuriot. Neurosymbolic AI for Reasoning over Knowledge Graphs: A Survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. <https://doi.org/10.1109/TNNLS.2024.3420218>.

- [3] Sen Yang, Xin Li, Leyang Cui, Lidong Bing, and Wai Lam. Neuro-Symbolic Integration Brings Causal and Reliable Reasoning Proofs. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- [4] Sanskar Sehgal and Yanhong A. Liu. Logical Lease Litigation: Prolog and LLMs for Rental Law Compliance in New York. In *Proceedings of the International Conference on Logic Programming (ICLP 2024)*, volume 416 of EPTCS, pages 59–68, 2025. <https://doi.org/10.4204/EPTCS.416.4>.
- [5] Model Context Protocol Contributors. Model Context Protocol Servers. <https://github.com/modelcontextprotocol/servers>, 2025. Accessed: 2025-07-22.
- [6] Jan Wielemaker and others. SWI-Prolog: A Modern Prolog System. <https://www.swi-prolog.org/>, 2025. Accessed: 2025-07-22.
- [7] Zenodo Community. Knowledge Graphs as Grounded World Models for Neuro-Symbolic Artificial Intelligence. <https://zenodo.org/records/18880228>, 2025. Accessed: 2025-07-22.
- [8] Adam Rybiński. Prolog MCP Server: Neurosymbolic AI for Modern Workflows. <https://dev.to/adamrybinski/prolog-mcp-server-neurosymbolic-ai-for-modern-workflows-3e35>, 2025. Accessed: 2025-07-22.
- [9] Yung-Shen Hsia, Fang Yu, and Jie-Hong Roland Jiang. Neuro-Symbolic Compliance: Integrating LLMs and SMT Solvers for Automated Financial Legal Analysis. *Proceedings of the 2nd ACM AIware Conference*, 2026.
- [10] Davide Gosmar and Deborah A. Dahl. Hallucination Mitigation with Agentic AI, Nested Learning, and AI Sustainability via Semantic Caching. *arXiv preprint*, 2026.
- [11] Davide Gosmar and Deborah A. Dahl. Hallucination Mitigation using Agentic AI Natural Language-Based Frameworks. *arXiv preprint*, 2025. Introduces the Total Hallucination Score (THS) KPIs.
- [12] Agnieszka Mensfelt, Adarsh Prabhakaran, Adrian Haret, Vince Trencsenyi, and Kostas Stathis. PrologMCP: A Standardized Prolog Tool Interface for LLM Agents. *arXiv preprint*, 2026. Accepted at Joint

Workshop on Statistics and Knowledge Integration for Logic, Learning,
Ethical Decisions, and LLMs, 18 July 2026, Lisbon.